

## 4. Data Preprocessing

1- Load the dataset and report its shape:

```
df = pd.read_csv(path, header=None, names=columns)

rows, cols = df.shape
print(f"Rows: {rows}, Columns: {cols}")

stats = df.drop("class", axis=1).agg(["mean", "std"])
print(stats)
print(df.head())
```

2- Compute and report the mean and standard deviation of each feature

```
Rows: 178, Columns: 14
      alcohol  malic_acid      ash  alkalinity_of_ash  magnesium \
mean  13.000618    2.336348  2.366517         19.494944   99.741573
std    0.811827    1.117146  0.274344         3.339564   14.282484

      total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins \
mean         2.295112    2.029270         0.361854         1.590899
std          0.625851    0.998859         0.124453         0.572359

      color_intensity      hue  od280_od315      proline
mean         5.058090  0.957449    2.611685  746.893258
std          2.318286  0.228572    0.709990  314.907474
```

3- Verify the absence of missing values

```
df.info()
print("Total number of missing values: ", df.isnull().sum().sum())
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   class                 178 non-null   int64
1   alcohol               178 non-null   float64
2   malic_acid            178 non-null   float64
3   ash                   178 non-null   float64
4   alkalinity_of_ash     178 non-null   float64
5   magnesium             178 non-null   int64
6   total_phenols         178 non-null   float64
7   flavanoids            178 non-null   float64
8   nonflavanoid_phenols  178 non-null   float64
9   proanthocyanins       178 non-null   float64
10  color_intensity       178 non-null   float64
11  hue                   178 non-null   float64
12  od280_od315           178 non-null   float64
13  proline               178 non-null   int64
dtypes: float64(11), int64(3)
memory usage: 19.6 KB
Total number of missing values: 0
```

#### 4- Create three versions of the dataset

A- Raw data: the df we already have

B- Z-score normalized data:

```
features = df.drop("class", axis=1)
# compute z-score
df_z = (features - features.mean()) / features.std()
df_z["class"] = df["class"]
print(df_z.head())
```

|   | alcohol  | malic_acid | ash       | alcalinity_of_ash | magnesium | \ |
|---|----------|------------|-----------|-------------------|-----------|---|
| 0 | 1.514341 | -0.560668  | 0.231400  | -1.166303         | 1.908522  |   |
| 1 | 0.245597 | -0.498009  | -0.825667 | -2.483841         | 0.018094  |   |
| 2 | 0.196325 | 0.021172   | 1.106214  | -0.267982         | 0.088110  |   |
| 3 | 1.686791 | -0.345835  | 0.486554  | -0.806975         | 0.928300  |   |
| 4 | 0.294868 | 0.227053   | 1.835226  | 0.450674          | 1.278379  |   |

|   | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | \ |
|---|---------------|------------|----------------------|-----------------|---|
| 0 | 0.806722      | 1.031908   | -0.657708            | 1.221438        |   |
| 1 | 0.567048      | 0.731565   | -0.818411            | -0.543189       |   |
| 2 | 0.806722      | 1.212114   | -0.497005            | 2.129959        |   |
| 3 | 2.484437      | 1.462399   | -0.979113            | 1.029251        |   |
| 4 | 0.806722      | 0.661485   | 0.226158             | 0.400275        |   |

|   | color_intensity | hue       | od280_od315 | proline   | class |
|---|-----------------|-----------|-------------|-----------|-------|
| 0 | 0.251009        | 0.361158  | 1.842721    | 1.010159  | 1     |
| 1 | -0.292496       | 0.404908  | 1.110317    | 0.962526  | 1     |
| 2 | 0.268263        | 0.317409  | 0.786369    | 1.391224  | 1     |
| 3 | 1.182732        | -0.426341 | 1.180741    | 2.328007  | 1     |
| 4 | -0.318377       | 0.361158  | 0.448336    | -0.037767 | 1     |

```
df_z.mean()
df_z.std()
```

|                      |          |
|----------------------|----------|
| alcohol              | 1.000000 |
| malic_acid           | 1.000000 |
| ash                  | 1.000000 |
| alcalinity_of_ash    | 1.000000 |
| magnesium            | 1.000000 |
| total_phenols        | 1.000000 |
| flavanoids           | 1.000000 |
| nonflavanoid_phenols | 1.000000 |
| proanthocyanins      | 1.000000 |
| color_intensity      | 1.000000 |
| hue                  | 1.000000 |
| od280_od315          | 1.000000 |
| proline              | 1.000000 |
| class                | 0.775035 |
| dtype:               | float64  |

### C- Min-Max normalized data (scaling on 0,1)

```
features = df.drop("class", axis=1)
# apply min-max scaling
df_minmax = (features - features.min()) / (features.max() - features.min())
df_minmax["class"] = df["class"]
print(df_minmax.head())
```

|   | alcohol  | malic_acid | ash      | alcalinity_of_ash | magnesium | \ |
|---|----------|------------|----------|-------------------|-----------|---|
| 0 | 0.842105 | 0.191700   | 0.572193 | 0.257732          | 0.619565  |   |
| 1 | 0.571053 | 0.205534   | 0.417112 | 0.030928          | 0.326087  |   |
| 2 | 0.560526 | 0.320158   | 0.700535 | 0.412371          | 0.336957  |   |
| 3 | 0.878947 | 0.239130   | 0.609626 | 0.319588          | 0.467391  |   |
| 4 | 0.581579 | 0.365613   | 0.807487 | 0.536082          | 0.521739  |   |

|   | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | \ |
|---|---------------|------------|----------------------|-----------------|---|
| 0 | 0.627586      | 0.573840   | 0.283019             | 0.593060        |   |
| 1 | 0.575862      | 0.510549   | 0.245283             | 0.274448        |   |
| 2 | 0.627586      | 0.611814   | 0.320755             | 0.757098        |   |
| 3 | 0.989655      | 0.664557   | 0.207547             | 0.558360        |   |
| 4 | 0.627586      | 0.495781   | 0.490566             | 0.444795        |   |

|   | color_intensity | hue      | od280_od315 | proline  | class |
|---|-----------------|----------|-------------|----------|-------|
| 0 | 0.372014        | 0.455285 | 0.970696    | 0.561341 | 1     |
| 1 | 0.264505        | 0.463415 | 0.780220    | 0.550642 | 1     |
| 2 | 0.375427        | 0.447154 | 0.695971    | 0.646933 | 1     |
| 3 | 0.556314        | 0.308943 | 0.798535    | 0.857347 | 1     |
| 4 | 0.259386        | 0.455285 | 0.608059    | 0.325963 | 1     |

```
print("min:\n", df_minmax.min(), "\n")
print("max:\n", df_minmax.max(), "\n")
```

```
min:
alcohol      0.0
malic_acid   0.0
ash          0.0
alcalinity_of_ash  0.0
magnesium    0.0
total_phenols 0.0
flavanoids   0.0
nonflavanoid_phenols 0.0
proanthocyanins 0.0
color_intensity 0.0
hue          0.0
od280_od315  0.0
proline      0.0
class        1.0
dtype: float64

max:
alcohol      1.0
malic_acid   1.0
ash          1.0
alcalinity_of_ash  1.0
magnesium    1.0
total_phenols 1.0
flavanoids   1.0
...
proline      1.0
class        3.0
dtype: float64
```

## 5. Exploratory Data Analysis (EDA) and Visualization

a. Generate a summary statistics table for all numerical features

```
summary = df.drop("class", axis=1).describe().T
print(summary)
```

|                      | count | mean       | std        | min    | 25%      | 75%      | max     |
|----------------------|-------|------------|------------|--------|----------|----------|---------|
| alcohol              | 178.0 | 13.000618  | 0.811827   | 11.03  | 12.3625  | 13.6775  | 14.83   |
| malic_acid           | 178.0 | 2.336348   | 1.117146   | 0.74   | 1.6025   | 3.0825   | 5.80    |
| ash                  | 178.0 | 2.366517   | 0.274344   | 1.36   | 2.2100   | 2.5575   | 3.23    |
| alcalinity_of_ash    | 178.0 | 19.494944  | 3.339564   | 10.60  | 17.2000  | 21.5000  | 30.00   |
| magnesium            | 178.0 | 99.741573  | 14.282484  | 70.00  | 88.0000  | 107.0000 | 162.00  |
| total_phenols        | 178.0 | 2.295112   | 0.625851   | 0.98   | 1.7425   | 2.8000   | 3.88    |
| flavanoids           | 178.0 | 2.029270   | 0.998859   | 0.34   | 1.2050   | 2.8750   | 5.08    |
| nonflavanoid_phenols | 178.0 | 0.361854   | 0.124453   | 0.13   | 0.2700   | 0.4375   | 0.66    |
| proanthocyanins      | 178.0 | 1.590899   | 0.572359   | 0.41   | 1.2500   | 1.9500   | 3.58    |
| color_intensity      | 178.0 | 5.058090   | 2.318286   | 1.28   | 3.2200   | 6.2000   | 13.00   |
| hue                  | 178.0 | 0.957449   | 0.228572   | 0.48   | 0.7825   | 1.1200   | 1.71    |
| od280_od315          | 178.0 | 2.611685   | 0.709990   | 1.27   | 1.9375   | 3.1700   | 4.00    |
| proline              | 178.0 | 746.893258 | 314.907474 | 278.00 | 500.5000 | 985.0000 | 1680.00 |

Every feature has **178.0** count

That strongly indicates **no missing values**. Clean dataset

### Feature Scales (Very Important)

variables are on **very different scales**:

- hue → mean ≈ **0.96**
- nonflavanoid\_phenols → mean ≈ **0.36**
- alcohol → mean ≈ **13**
- magnesium → mean ≈ **100**
- proline → mean ≈ **747**

proline ranges from **278 to 1680** — huge compared to others.

This dataset **definitely needs feature scaling**

## Spread & Variability

standard deviations:

- proline → **314.9** (very high variability)
- color\_intensity → **2.31**
- flavanoids → **0.99**
- ash → **0.27** (relatively tight spread)

Some features vary a lot more than others — again reinforcing the need for scaling

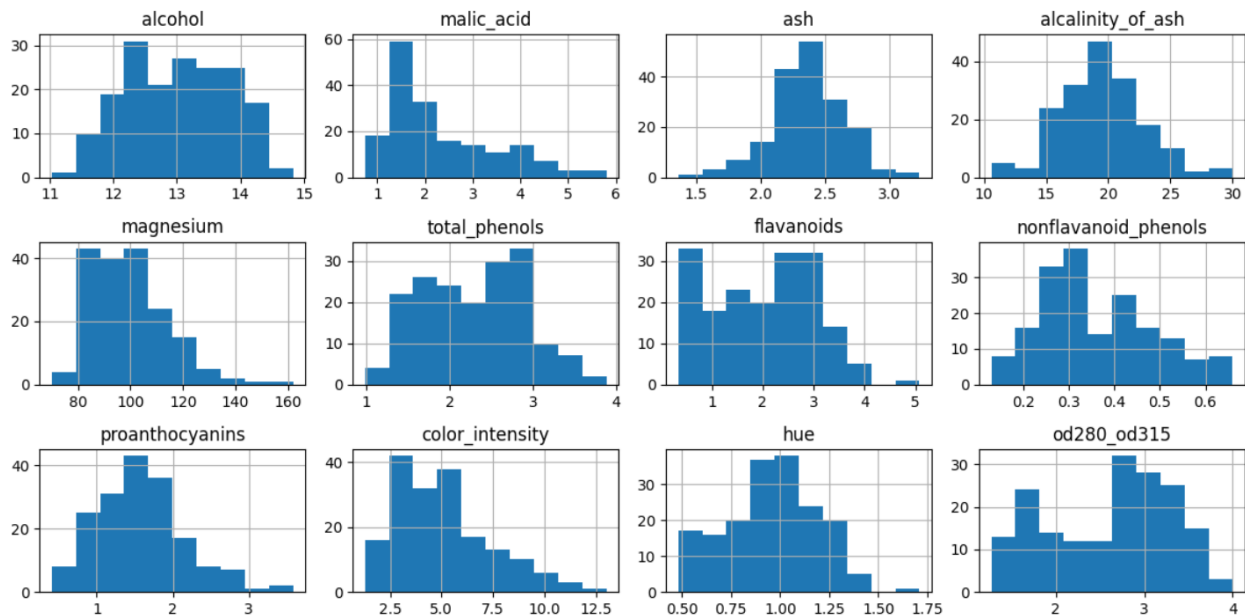
## Potential Skewness / Outliers

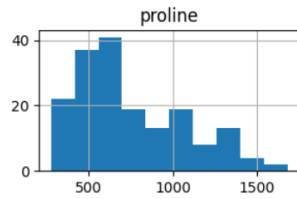
Compare median (50%) vs mean:

- proline: mean = 746, median = 673  
→ slightly right-skewed
- color\_intensity: mean = 5.05, median = 4.69, max = 13  
→ likely right-skewed with some high-value wines
- flavanoids: max = 5.08, Q3 = 2.87  
→ possible high-end outliers

So yes — some features likely have **right skew / high-value tails**

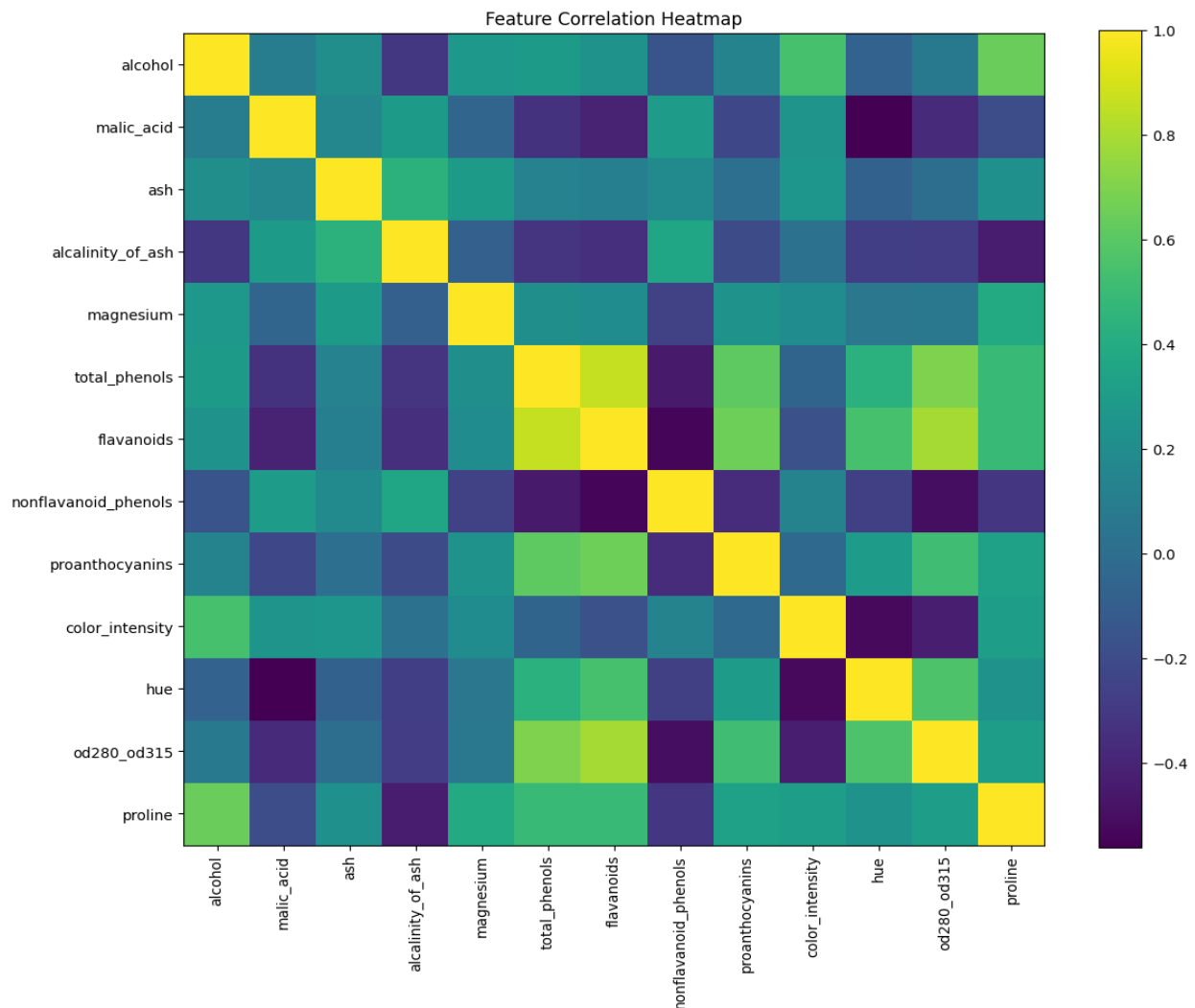
b. Plot histograms for at least five selected features





As already indicated from the summary, it can be seen from the histograms that **proline** and **color\_intensity**, and **magnesium** are both right-skewed; The one that are almost normal is the **alcalinity\_of\_ash**.

c. Compute and visualize the correlation matrix using a heatmap



- +1 → strong positive relation
- 0 → independent
- -1 → strong negative relation

```

corr = df.drop("class", axis=1).corr().abs()
np.fill_diagonal(corr.values, 0)

# flatten to series
corr_flat = corr.unstack()

# sort descending
corr_sorted = corr_flat.sort_values(ascending=False)

# get top 2 pairs (ignore duplicates like (A,B) and (B,A))
seen = set()
top_pairs = []
for (f1, f2), val in corr_sorted.items():
    if (f2, f1) not in seen:
        top_pairs.append((f1, f2, val))
        seen.add((f1, f2))
    if len(top_pairs) == 2:
        break

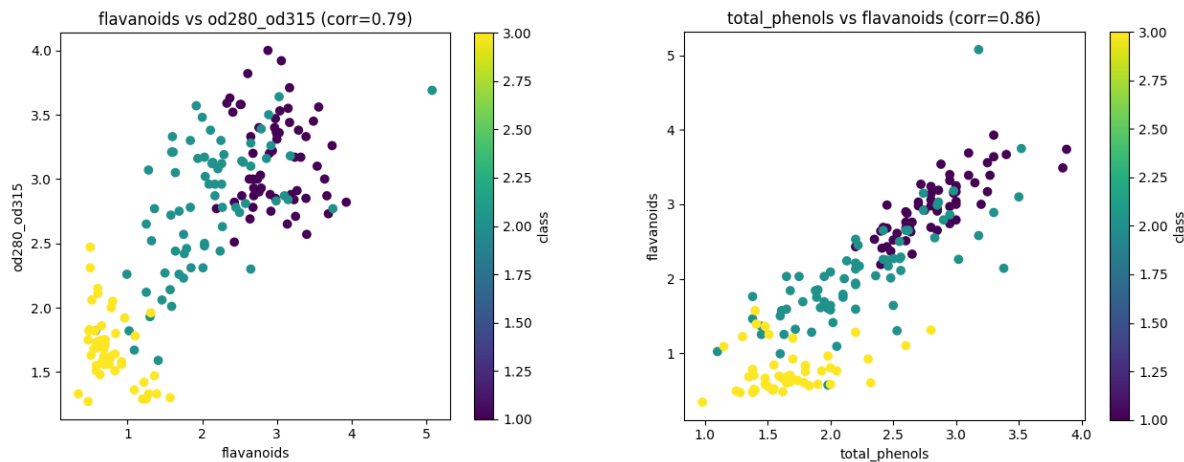
top_pairs

[('total_phenols', 'flavanoids', 0.8645635000951147),
 ('flavanoids', 'od280_od315', 0.787193901866951)]

```

After sorting the values of the heatmap, the highest correlation is a positive correlation between the pairs: [total\\_phenols, flavanoids](#) and [flavanoids, od280\\_od315](#)

d. Create scatter plots for at least two meaningful pairs of features



The ones with highest correlation, therefore the scatter plot shows a linear relation between the attributes.

## 5.2 Dimensionality Reduction for Visualization

```
X = df_z.drop("class", axis=1).values # shape: (178, 13)
y = df_z["class"].values

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X) # shape: (178, 2)

print("Explained variance ratio:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,6))
scatter = plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='viridis', edgecolor='k')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Wine Dataset (2D)")
plt.colorbar(scatter, label="class")
plt.tight_layout()
plt.show()
```

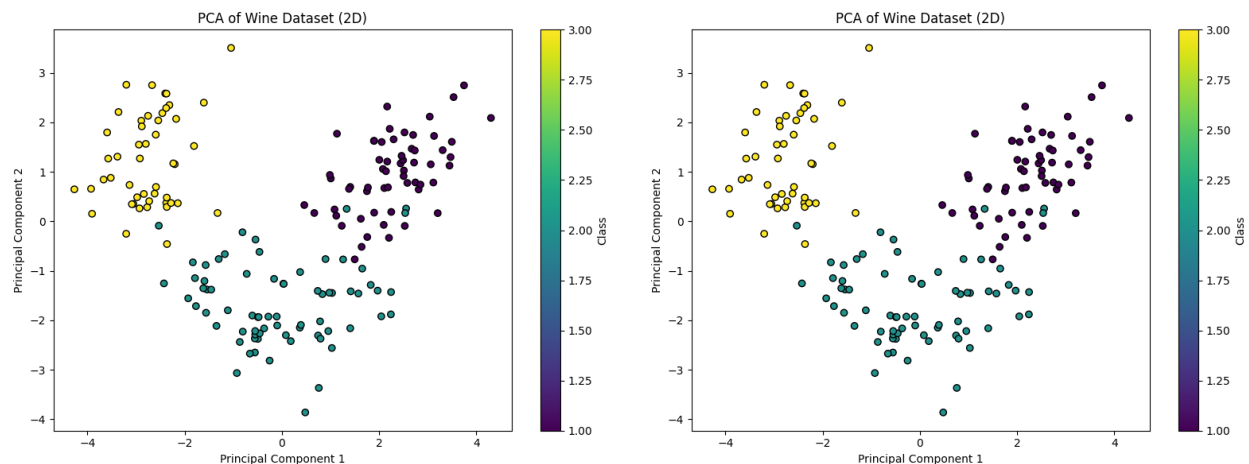
Explained variance ratio: [0.36198848 0.1920749 ]

Principal Component Analysis (PCA) was applied to reduce the dimensionality of the dataset from 13 features to 2 principal components. The first component explains 36.2% of the total variance, while the second explains 19.2%, resulting in a combined explained variance of approximately 55.4%. This indicates that more than half of the dataset's variability is preserved in the two-dimensional representation, providing a reasonable low-dimensional visualization while retaining a substantial portion of the original information.

Notice. Because **PCA is variance-based**, and variance depends on scale

➔ we must use the normalized data to apply PCA on.

The PCA algorithm was applied on z-score normalized data, I also tried it on min-Max normalized data and it wasn't that different in terms of the variance ratio and overall correlation of the dimensions.





## 6. Part 1: Distance Behavior in High-Dimensional Space

```
X = df_z.drop("class", axis=1).values
# Euclidean distances
dist_euclidean = pdist(X, metric='euclidean')

# Manhattan distances
dist_manhattan = pdist(X, metric='cityblock')

# Cosine distances
dist_cosine = pdist(X, metric='cosine')

for name, d in [("Euclidean", dist_euclidean),
                ("Manhattan", dist_manhattan),
                ("Cosine", dist_cosine)]:
    print(f"{name} distances:")
    print(f"Mean: {d.mean():.3f}")
    print(f"Std : {d.std():.3f}")
    print(f"Min : {d.min():.3f}")
    print(f"Max : {d.max():.3f}")
    print("-"*30)
```

Euclidean distances:

Mean: 4.892  
Std : 1.437  
Min : 1.161  
Max : 11.180

-----  
Manhattan distances:

Mean: 14.590  
Std : 4.693  
Min : 3.187  
Max : 31.911

-----  
Cosine distances:

Mean: 1.004  
Std : 0.437  
Min : 0.045  
Max : 1.918  
-----

Pairwise distance analysis in the 13-dimensional feature space shows **moderate dispersion** among samples.

The mean Euclidean distance is 4.89 ( $\pm 1.44$ ), indicating that observations are **reasonably separated but not excessively dispersed**.

Manhattan distances scale consistently with dimensionality, while the average cosine distance of approximately 1.0 suggests that sample vectors are roughly orthogonal on average. Overall, the dataset exhibits meaningful geometric variation without severe distance concentration effects.

## 7. Part 2: Impact of Feature Scaling on K-Means

```
datasets = {
    "Raw": df.drop("class", axis=1).values,
    "Z-score": df_z.drop("class", axis=1).values,
    "Min-Max": df_minmax.drop("class", axis=1).values
}

results = []

for name, X in datasets.items():
    kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)

    inertia = kmeans.inertia_
    silhouette = silhouette_score(X, labels)

    results.append({
        "Dataset": name,
        "Inertia": inertia,
        "Silhouette": silhouette
    })

results_df = pd.DataFrame(results)
print(results_df)
```

✓ 5.7s

|   | Dataset | Inertia      | Silhouette |
|---|---------|--------------|------------|
| 0 | Raw     | 2.370690e+06 | 0.571138   |
| 1 | Z-score | 1.270749e+03 | 0.284859   |
| 2 | Min-Max | 4.896052e+01 | 0.300894   |

### Inertia (Within-Cluster Sum of Squares)

- Used mostly with **k-means**. It measures **how tightly the points in a cluster are grouped around the cluster center**
- Smaller inertia → points are closer to their cluster centers → tighter clusters
- Larger inertia → clusters are more spread out → less compact
- Important: **inertia always decreases if you increase the number of clusters**, even if the new clusters aren't meaningful. So it's not enough by itself to pick k

### Silhouette Score

- Measures **how well-separated the clusters are**, considering both cohesion (within-cluster) and separation (between clusters)
- a = average distance to other points in the same cluster (intra-cluster distance)
- b = average distance to points in the nearest other cluster (nearest-cluster distance)
- **+1** → sample is well-matched to its own cluster and far from others (good)
- **0** → sample is on or very close to the decision boundary between clusters
- **-1** → sample may be in the wrong cluster

$$\text{Inertia} = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

$$s = \frac{b - a}{\max(a, b)}$$

| Dataset | Inertia   | Silhouette |
|---------|-----------|------------|
| Raw     | 2,370,690 | 0.571      |
| Z-score | 1,270     | 0.285      |
| Min-Max | 48.96     | 0.301      |

Raw data:

- Very high inertia → makes sense because your features are on wildly different scales (proline dominates, hue is tiny).
- Silhouette  $\approx 0.57$  → actually **not bad**. Shows clusters are reasonably separated

Standardized data:

- Inertia dropped massively (because data now has unit variance). That's expected
- Silhouette **also dropped a lot** → clusters are **less well-separated in normalized space**
  - K-means is now treating all features equally:
  - Features like proline no longer dominate
  - Small chemical differences get equal weight
  - Clusters that were obvious in raw scale are now blurred

Min-Max scaled data:

- Again, inertia is tiny (scaling effect).
- Silhouette slightly better than z-score, but still low → clusters are **not very distinct** in this space

Overall interpretation:

- On **raw data**, k-means is "seeing" the few big-scale features (proline, magnesium) → forms clusters largely along them. That's why silhouette was higher
- On **normalized data**, all features contribute equally → cluster boundaries blur → silhouette drops
- Absolute inertia doesn't tell much without context (it depends on scale). Silhouette is more meaningful
- Silhouette  $\approx 0.28$ – $0.30$  → **poor clustering**. 0.5+ is usually decent;  $>0.7$  is strong

## 8. Part 3: K-Means Implementation from Scratch

| Dataset | Method               | Inertia      | Iterations | Runtime (s) |
|---------|----------------------|--------------|------------|-------------|
| Raw     | Custom K-Means       | 2,370,689.69 | 10         | 0.0034      |
| Raw     | scikit-learn K-Means | 2,370,689.69 | 6          | 0.0144      |
| Z-score | Custom K-Means       | 1,270.75     | 10         | 0.0061      |
| Z-score | scikit-learn K-Means | 1,270.75     | 7          | 0.0195      |
| Min-Max | Custom K-Means       | 48.97        | 8          | 0.0031      |
| Min-Max | scikit-learn K-Means | 48.96        | 4          | 0.0205      |

- Lower inertia → tighter clusters:
  - Min-Max scaled dataset has the **lowest inertia** (~48.96–48.97)
- Fewer iterations → faster convergence:
- scikit-learn K-Means generally converges faster than custom, especially for scaled data:
  - Min-Max scikit-learn: 4 iterations (fewest overall)
- Custom K-Means is slightly faster on small datasets (0.003–0.006 s):
  - Scikit-learn is slightly slower on small datasets (0.014–0.020 s), but scales better for large datasets

For  $k = 3$ , K-Means clustering was applied to the dataset under three preprocessing conditions: raw, Z-score normalized, and Min-Max scaled. Across all datasets, both the custom implementation and scikit-learn's K-Means converged to nearly identical inertia values, indicating consistent clustering solutions. The number of iterations required for convergence varied slightly, with scikit-learn generally requiring fewer iterations due to optimized initialization, while runtime differences were minor and dataset-dependent. Notably, the inertia decreases substantially after normalization, reflecting the effect of scaling on the magnitude of within-cluster variance, but the relative clustering structure remains consistent across implementations.

## 9. Part 4: Hierarchical Clustering Stability Analysis

Cophentic:

- Imagine building a dendrogram (hierarchical tree)
- For any pair of samples, trace up the tree until their branches meet
- The vertical distance at that point = **cophenetic distance**
- ~1: Dendrogram faithfully preserves original distances → good clustering representation
- ~0: Dendrogram does not preserve distances well → clustering may not reflect true structure

| Rank | Linkage Method | Cophenetic Correlation |
|------|----------------|------------------------|
| 1    | average        | 0.7591                 |
| 2    | ward           | 0.6623                 |
| 3    | complete       | 0.5917                 |
| 4    | single         | 0.5436                 |

The **average** method is the most stable one as it provides the **highest cophentic** correlation, meaning the cluster distances on the tree is higher than the other methods, meaning the clustering is working quite well!

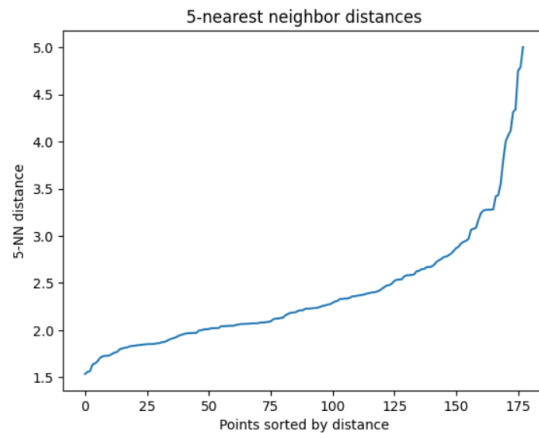
## 10. Part 5: Automated Density-Based Clustering

Density-based clustering groups together points that are **densely packed**, and separates areas of **low density** as noise or outliers

Unlike k-means, you **don't need to predefine the number of clusters**

**Key concepts (DBSCAN as example)**

- **$\epsilon$  (epsilon):** radius around a point to look for neighbors
- **MinPts:** minimum number of points required within  $\epsilon$  to form a dense region
- **Core points:** points with  $\geq$  MinPts neighbors in  $\epsilon$ -radius
- **Border points:** points in  $\epsilon$ -radius of a core point but with  $<$  MinPts neighbors
- **Noise points:** points that are neither core nor border



- Estimated epsilon: 3.0048055712308877
- Detected clusters: 1
- Noise points: 10

Interpretation:

- When  $\epsilon$  is too big: most points fall into a single dense region  $\rightarrow$  DBSCAN merges everything  $\rightarrow$  you get **1 cluster**
- DBSCAN only detects clusters if there are **dense regions separated by empty space**.
- Your 13D wine data is **relatively uniform in density** after Z-score normalization  $\rightarrow$  DBSCAN struggles

Solution: PCA to reduce the dimensionality

- Estimated epsilon: 0.675159736194372
- Detected clusters: 2
- Noise points: 7

Interpretation:

- 7 points are labeled as **outliers** because they didn't have enough neighbors within  $\epsilon$
- **DBSCAN works well after PCA or other dimensionality reduction** when high-dimensional data is too sparse.
- Having 2 clusters + a few noise points is realistic for the wine dataset; it reflects real separations in the main chemical profiles

## 11. Part 6: Fully Automated Clustering System

Result on standardized data:

```
{'Best Algorithm': 'DBSCAN', 'Hyperparameters': {'eps': np.float64(0.5), 'min_samples': 7},  
'Silhouette Score': 0.6502464641416931, 'Number of Clusters': 5, 'Number of Noise Points':  
np.int64(55)}
```

Result after applying PCA manually:

```
{'Best Algorithm': 'KMeans', 'Hyperparameters': {'n_clusters': 3, 'random_state': 42}, 'Silhouette  
Score': 0.5999742419909309, 'Number of Clusters': 3, 'Number of Noise Points': 0}
```

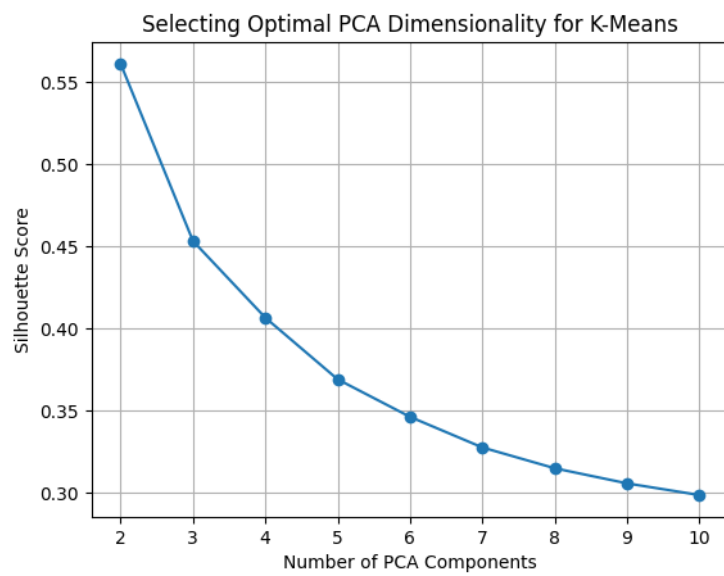
I added this evaluation function:

```
def evaluate_dbscan(X, eps, min_samples=5, apply_pca=False):  
    X_proc = preprocess_data(X, apply_pca=apply_pca)  
    labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X_proc)  
  
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)  
    n_noise = np.sum(labels == -1)  
    noise_ratio = n_noise / len(labels)  
  
    # Check if DBSCAN result is meaningful  
    if n_clusters < MIN_CLUSTERS or noise_ratio > MAX_NOISE_RATIO:  
        if not apply_pca:  
            # Retry with PCA  
            return evaluate_dbscan(X, eps, min_samples, apply_pca=True)  
        else:  
            # Already applied PCA, return what we have  
            return labels, n_clusters, n_noise, noise_ratio  
    return labels, n_clusters, n_noise, noise_ratio
```

It checks whether the clustering results are meaningful, if they're not, it applies PCA to reduce dimensionality and improve the clustering results.

### 12.1. Bonus 1: Subspace Clustering via PCA

| PCA Dimensions | Silhouette Score |
|----------------|------------------|
| 2              | 0.561            |
| 3              | 0.453            |
| 4              | 0.407            |
| 5              | 0.369            |
| 6              | 0.346            |
| 7              | 0.328            |
| 8              | 0.315            |
| 9              | 0.306            |
| 10             | 0.299            |



The highest score occurs when the number of components is 2.